Лабораторная работа № 15

Тема: HTTP-запросы: XMLHttpRequest и fetch.

Цель: Научиться отправлять асинхронные запросы к API, используя классический объект XMLHttpRequest и современный метод fetch, а также обрабатывать полученные данные в формате JSON.

Вариант 1

Задание 1. Классика: XMLHttpRequest
Используя объект XMLHttpRequest, сделайте GET-запрос к https://jsonplaceholder.typicode.com/posts/1. Выведите в консоль заголовок (title) полученного поста.
Требование: Обработайте событие onload и используйте JSON.parse().

Задание 2. Переход на fetch
Выполните то же самое задание (получение поста №1), но с использованием метода fetch. Выведите title в консоль. Сравните объем кода с первым заданием.

Задание 3. Цепочка данных (Chaining)
Используя fetch, получите данные пользователя (/users/2). Затем, используя userId из первого ответа, сделайте второй запрос, чтобы получить список всех альбомов этого пользователя (/albums?userId=2). Выведите количество найденных альбомов.

Задание 4. Работа с текстом
Сделайте запрос к https://httpbin.org/robots.txt. Используйте метод .text() вместо .json(), чтобы прочитать содержимое файла и вывести его в консоль.

Задание 5. Обработка сетевой ошибки
Напишите запрос fetch к заведомо несуществующему домену (например, https://not-exists-12345.com). Используйте метод .catch(), чтобы вывести в консоль сообщение: «Произошла ошибка соединения».

Вариант 2

Задание 1. Классика: XMLHttpRequest
Используя XMLHttpRequest, получите список всех пользователей (/users). Выведите в консоль имя (name) только первого пользователя из массива.

Требование: Настройте request.responseType = 'json', чтобы не вызывать JSON.parse() вручную.

Задание 2. fetch и стрелочные функции
Используя fetch, получите данные о задаче (/todos/5). Выведите в консоль текст задачи (title) и статус её выполнения (completed). Используйте краткую запись стрелочных функций.

Задание 3. Связанные запросы
Используя fetch, получите данные о конкретном комментарии (/comments/1). Получите из него postId. Сделайте второй запрос к самому посту (/posts/{postId}) и выведите в консоль его заголовок.

Задание 4. Проверка статуса (response.ok)
Сделайте запрос к https://jsonplaceholder.typicode.com/invalid-page. В первом .then() проверьте свойство response.ok. Если оно false, выведите в консоль сообщение: «Ошибка: страница не найдена (404)».

Задание 5. Случайные данные (Dog API)
Используя fetch, получите случайную картинку собаки с адреса https://dog.ceo/api/breeds/image/random. Выведите в консоль только строку с URL-адресом картинки (поле message в полученном JSON).

Шпаргалка: fetch, async/await и статус ответа
1. Простейший запрос с выводом статуса

```
async function demo() {
  let response = await
fetch("https://jsonplaceholder.typicode.com/todos/1");
  console.log("Статус ответа:", response.status);

  let data = await response.json();
  console.log("Данные:", data);
}

demo();
```

📌 Здесь видно:
response.status → числовой код (200, 404 и т. д.),
данные преобразуются в JSON.

2. Проверка статуса через ok

```
async function checkStatus() {
  let response = await
fetch("https://jsonplaceholder.typicode.com/unknown");
  console.log("Статус ответа:", response.status);

  if (!response.ok) {
    console.error("Ошибка HTTP:", response.status);
    return;
  }

  let data = await response.json();
  console.log("Данные:", data);
}

checkStatus();
```

📌 Если сервер вернёт 404, мы это заметим через ok.

3. Сетевая ошибка (try/catch)

```
async function networkFail() {
  try {
    let response = await fetch("https://wrong-
domain.example.com/data");
    console.log("Статус ответа:", response.status); // сюда
не дойдёт
    let data = await response.json();
    console.log("Данные:", data);
  } catch (error) {
    console.error("Сетевая ошибка:", error.message);
  }
}

networkFail();
```

📌 Ошибки сети (например, нет интернета) ловятся только через catch.

4. Работа с текстом, а не JSON

```
async function getText() {
  let response = await fetch("https://httpbin.org/html");
  console.log("Статус ответа:", response.status);
```

```
  let text = await response.text();
  console.log("HTML длиной:", text.length, "символов");
}

getText();
```

📌 У объекта Response можно вызывать не только .json(), но и .text(), .blob(), .arrayBuffer().

5. Универсальная функция для запросов

```
async function fetchData(url) {
  try {
    let response = await fetch(url);
    console.log("Статус:", response.status);

    if (!response.ok) {
      throw new Error("Ошибка HTTP " + response.status);
    }

    let data = await response.json();
    return data;
  } catch (error) {
    console.error("Ошибка запроса:", error.message);
  }
}
```

// Пример использования:

```
fetchData("https://jsonplaceholder.typicode.com/todos/2")
  .then(data => console.log("Задача:", data));
```

📌 Такая функция помогает не дублировать проверку статуса в каждом запросе.

⚡ **Мини-таблица полезных статусов HTTP**

200 — OK (успешно).
201 — Created (создано, обычно для POST).
400 — Bad Request (ошибка клиента).
401 — Unauthorized (нужна авторизация).
403 — Forbidden (доступ запрещён).

404 — Not Found (ресурс не найден).
500 — Internal Server Error (ошибка сервера).

Используйте response.status для проверки кода.
response.ok помогает быстро понять, успешный ли запрос.
Сетевые ошибки ловятся только в try/catch.
Помимо .json() есть и другие методы для чтения ответа.
Универсальная функция fetchData экономит время и уменьшает ошибки.